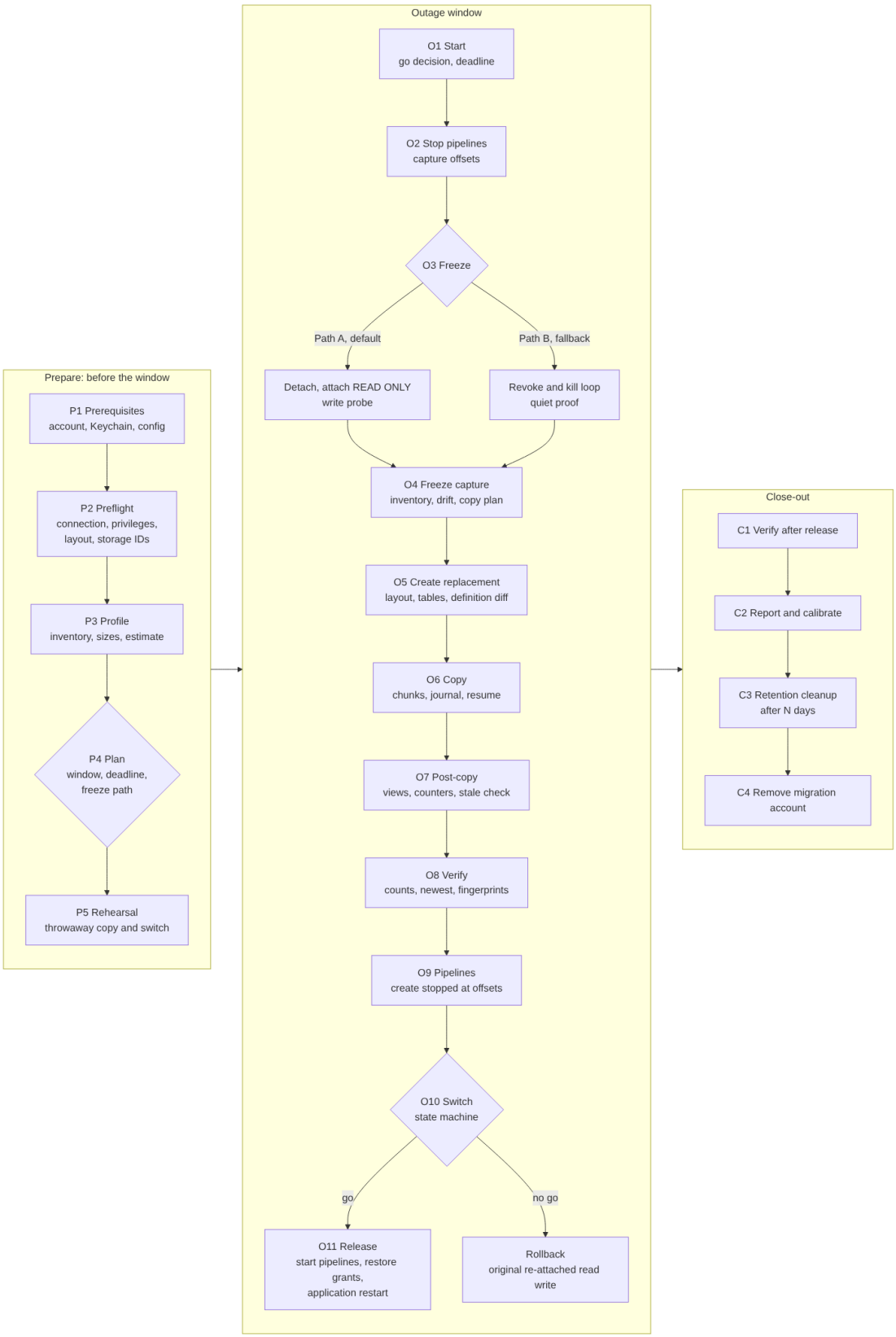
SingleStore Database Rebuild Kit

Step guide: the README and the step catalog

Author: John Bergamini

Version 1.0.0

Flow of the steps



SingleStore Database Rebuild Kit

Author: John Bergamini. Version 1.0.0. Proprietary; see LICENSE.

The kit rebuilds a SingleStore Helios database into a new partition layout and cuts it over under its original name inside one outage window. The first outage step makes the source read only. Everything after that is a copy, a verification and a switch, each behind a gate, each restartable from a terminal.

Who runs what

Identity	Who	Steps
runner	A SQL account the kit creates at P1 and drops at C4	P2, P3 tables, O4 to O8, C1, C2, watch
owner	The workspace admin SQL user, or the Portal identity in the owner notebook	P1 account, P3 pipelines, O2, O3, O7 views, O9, O10, O11, C3, C4, lockout
operator	The person running the kit	every command; records the gates a person decides
customer	The application owner	stops the application (if asked), hands over pipeline credentials, restarts the application at O11

The runner copies. The owner freezes, switches and releases. With an owner profile in the configuration the kit runs the admin steps itself; without one it prints each admin block and waits for `s2rebuild gate`.

Install

Python 3.10 or later on macOS or Linux. The Keychain password source is macOS only; elsewhere use `password_env`.

```
python3 -m venv .venv
.venv/bin/pip install -e .
.venv/bin/s2rebuild --help
```

Run the offline tests with `.venv/bin/pip install pytest` and `.venv/bin/python -m pytest`. No test opens a network connection.

Configure

1. Copy the example: `cp examples/config.example.yaml customer/config.yaml`.
2. Set the workspace host, the database names and the target layout. Every key is in docs/CONFIG.md.
3. Store each password in the macOS Keychain (`security add-generic-password -a "$USER" -s s2rebuild-runner -w`) or name an environment variable with `password_env`. The kit never prints or writes a password.
4. Everything a run writes stays under `customer/`. The rest of the folder is the static tree.

Steps

Each step prints what it did and the gate it needs. Exit codes: 0 done, 1 refused, 2 incomplete (rerun to resume), 3 verification mismatch, 4 gate pending or failed. The full catalog with inputs, outputs, statements and recovery is docs/STEPS.md.

Every command works on one run under `customer/runs/`. The P1 gate starts the run and prints its id. Export that id as `S2REBUILD_RUN`, so every later command names the same run. Without `S2REBUILD_RUN` or `--run ID`, a command uses the most recent run. `s2rebuild run` resumes that run and skips the steps that

passed. It starts a run only when none exists, or with `--new` (`--new --run ID` names the new run). Once a run exists, `s2rebuild run --run ID` refuses an id that names no run, so a mistyped id never starts another run. Every gate hint and every log header prints the run id with its `--run` form.

The sequence from P1 to the go and on to O2, where RUN_ID is the id the P1 gate printed:

```
s2rebuild account-sql
s2rebuild gate P1 --pass --note TEXT
export S2REBUILD_RUN=RUN_ID
s2rebuild check
s2rebuild profile
s2rebuild plan
s2rebuild gate P4 --pass --note TEXT
s2rebuild rehearse
s2rebuild run --from O1
s2rebuild gate O1 --pass --note TEXT
s2rebuild run
```

Prepare (before the window)

1. P1 Prerequisites. Print the runner account block with `s2rebuild account-sql` and have the admin run it, or run it with `s2rebuild account-sql --execute`. Store the passwords and write the configuration. Gate: the account, the Keychain items and the configuration are in place; record it with `s2rebuild gate P1 --pass --note TEXT`. When no run exists yet, this gate starts the run and prints its id; export it as `S2REBUILD_RUN`.
2. P2 Preflight. `s2rebuild check` connects as the runner (and the owner), checks every runner privilege by name, records the source STORAGE_ID, confirms the replacement and retained names are free and the compute size fits the partition count. It refuses until P1 is recorded. Run it again any time: O1 needs P2 passed in the same run. Gate: every line ok.
3. P3 Profile. `s2rebuild profile` captures the inventory (tables, keys, sizes, accounts, sessions, views, pipelines with the owner profile) and writes `profile.xlsx` with the estimate and the window fit. It refuses until P2 passed. Once P3 passed, a new capture needs `--redo`, which logs what it rewrites. `--dry-run` writes only `profile-dry-run.xlsx`, never the capture. Gate: the estimate fits the window. Procedures and functions are not migrated in this pass: `check` and `profile` refuse until `objects.routines.ignore` lists each one.
4. P4 Plan. `s2rebuild plan` prints the plan sheet: window, abort deadline, freeze path, step owners, pipeline credential route. Gate: the decisions are recorded with `s2rebuild gate P4 --pass --note TEXT`.
5. P5 Rehearsal. `s2rebuild rehearse` builds throwaway `s2rebuild_rehearsal_` databases, copies them with a SIGKILL test and a resume, verifies, freezes, switches and drops what it created. The killed chunk leaves a leaf lock for 15 to 20 minutes, so plan 30 minutes. Gate: P5 passed (the view at O7 and the switch need the owner profile).

Outage window

6. O1 Start. `s2rebuild run --from O1` checks that P2 passed in this run and that rehearsal evidence exists (P5 passed, or a passed rehearsal of any run), records the window, the abort deadline and the configuration hash, and waits for the go decision: `s2rebuild gate O1 --pass --note TEXT`. Without a rehearsal, `--no-rehearsal NOTE` starts anyway and records the note. Then `s2rebuild run` resumes the same run: it skips O1 and continues at O2. `s2rebuild run --new` starts another run instead, which needs its own P1 and P2.

7. O2 Stop pipelines. The owner stops every pipeline and the kit captures every definition and offset again after the stop. These are the offsets O9 replays. Gate: no pipeline Running; loaded and successful offsets agree on every partition; capture written.
8. O3 Freeze. Path A (default): DETACH, ATTACH READ ONLY, a write probe that must be refused. A DETACH that waits longer than `freeze.detach_wait_seconds` is cancelled and issued again with KILL. Path B (single database only): `s2rebuild lockout --hold` revokes write privileges and kills sessions until the quiet proof passes, then keeps passing until S1 of O10 has run; leave it running in a second terminal through O10. After a notebook or hand run, `s2rebuild step O3 --redo` runs the probe and records freeze_time. A run that stopped between DETACH and ATTACH resumes at the ATTACH: `s2rebuild step O3 --redo` sees the detached storage and the free source name and never detaches again. For a hand run, `s2rebuild owner-sql O3` writes that block to `owner-sql/O3-reentry.sql` beside `O3.sql`. Gate: probe refused and READONLY (A), or quiet proof passed (B).
9. O4 Freeze capture. `s2rebuild profile --step O4` captures the inventory after the freeze and reports drift against P3. It refuses until O1 to O3 passed. Once O4 passed, a new capture needs `--redo`: it rewrites the copy plan. Gate: drift reviewed.
10. O5 Create replacement. `s2rebuild run --from O4` continues: the replacement is created with the target layout and every table definition is compared with the capture; its STORAGE_ID is recorded.
11. O6 Copy. Server-side INSERT ... SELECT in hash buckets, journaled, resumable. Stop it only with `s2rebuild pause` or the abort deadline: a killed client leaves a prepared transaction holding a leaf table lock for 15 to 20 minutes. Gate: every chunk done; per-table count proof all match.
12. O7 Post-copy. Views, AUTO_INCREMENT counters and the stale check (counts and newest values) with re-copy of anything that moved. A completed re-copy marks O6 passed again, and the counters are set after the last re-copy. A view runs under its definer and C4 removes the runner, so the runner never creates one. With the owner profile the kit creates the views, keeping each original definer whose account still exists. Without it O7 writes `owner-sql/O7.sql` and ends pending: run that block as the admin, then `s2rebuild step O7`. A view whose definition names the source database refuses the step. Gate: views created, zero mismatches.
13. O8 Verify. Counts, newest values and grouped fingerprints within the time budget. Gate: tiers 1 and 2 pass for every table; tier 3 within budget or accepted with `--yes`. A mismatch marks the buckets for re-copy; run `s2rebuild run --from O6` to copy them again and rerun O7 and O8.
14. O9 Pipelines. Export the credential variables for this step only, run it, then unset them. The kit recreates every pipeline in the replacement, stopped, at the O2 offsets. Route customer: the customer runs the placeholder file and the kit checks. Gate: count, none Running, offsets equal the O2 capture partition by partition.
15. O10 Switch. A state machine: retained copy attached read only first, then the replacement as a branch under the live name. S0 refuses unless every planned table was verified after its last chunk was copied; without the owner profile O10 refuses on the same rule, and the printed block and the notebook cell start with a STOP while it holds. A re-copy rewrites `owner-sql/O10.sql` with a STOP; generate the blocks or the notebook again after O8 passes. On Path B, S0 and S1 refuse unless the last lockout pass was quiet within `freeze.quiet_max_age_seconds` (default 60); that refusal comes before the go, so a re-copy stays possible. The printed block and the notebook cell check instead, before the DETACH, that every privilege the lockout took is still revoked. Type the live name when asked (or pass `--yes`). Gate: the live name resolves to the new branch with the target layout and the capture counts. From S3 the live name accepts writes. A rollback (`s2rebuild owner-sql O10 --rollback`) runs only while every table still holds the row count of the copy and O11 has not started. The count check cannot see an update

that keeps the count, so a rollback after S3 is safe only when the application stayed stopped until O11, as the O1 note records.

16. O11 Release. Retention check per partition, grants restored, pipelines started; the customer restarts the application. Gate: application writing.

Close-out

17. C1 Verify after release. `s2rebuild step C1`: writer watch, count changes, pipelines Running with offsets advancing, application-managed pipelines back, no errors, grants compared. Gate: all ok and application writes seen. For a live database idle on purpose, pass `s2rebuild step C1 --idle-workload NOTE`. Without the owner profile the pipeline and grants checks are recorded as not checked.
18. C2 Report and calibrate. `s2rebuild step C2` writes `final.xlsx` and `run-report.md` and refits the throughput model into `customer/baseline.json`. With too few chunks to fit, the baseline in use stays, the reports say so, and C2 passes. When the kit cannot pass C1 or C2, record that step by hand with `s2rebuild gate ID --pass --note TEXT --override NOTE`.
19. C3 Retention cleanup. After `closeout.retain_days`: `s2rebuild step C3` drops the retained branch and the replacement. Each DROP runs only while the name holds the STORAGE_ID and attach type the run recorded. Another database under either name, or a recorded STORAGE_ID under another name, stops the step with nothing dropped. A rerun skips a name already dropped. The step passes only when neither recorded STORAGE_ID is attached anywhere. Delete the detached original in the Portal by STORAGE_ID.
20. C4 Remove the migration account. `s2rebuild step C4` drops the runner account; delete its Keychain item. The step stops before DROP USER while a view in the live database has the runner as its definer.

Troubleshooting

- Re-copy after O8: `s2rebuild mark-recopy TABLE [--bucket N]` sets O6, O7 and O8 back to pending and clears the table's verification. Run `s2rebuild run --from O6`; O6 copies the buckets again, then O7 and O8 run again. The switch refuses until O8 passes again. A fresh-table load reopens O7 and O8 the same way. Both rewrite `owner-sql/O10.sql` and `O10-rollback.sql` with a STOP; after O8 passes, run `s2rebuild owner-sql O10` (and `--rollback`) or `s2rebuild owner-notebook` again, and never run the O10 cells of an older notebook. Once the switch may have begun (O10 passed, the go recorded, an O10 gate, or the retained or live attachment recorded), a re-copy is refused; a new copy is a new run.
- Lock waits (error 1205): the kit serializes the table, probes the lock and retries after `copy.lock_retry_seconds`. A lock held by a killed client clears in 15 to 20 minutes. A chunk fails after `copy.max_lock_waits` lock waits and O6 stops INCOMPLETE with the command to run: `s2rebuild step O6 --fresh-table TABLE` loads the table into a new table and renames it. It refuses past the abort deadline or while the stop file exists. After a failure, rerun the same command; it resumes and does not load a complete fresh table again.
- Detach waits: a DETACH blocked by writers is cancelled after `freeze.detach_wait_seconds` and issued with KILL when `freeze.detach_kill` is true. With `detach_kill: false` the step stops; run `s2rebuild lockout` as a bridge, then `s2rebuild step O3 --redo`.
- Grants restored by another session: the lockout logs `GRANT RESTORED BY ANOTHER SESSION` and revokes again on the next pass, with `--hold` also after the quiet proof. Find the job that restores them before the night.

- Name collisions: names are not unique on Helios. Every ATTACH names a STORAGE_ID. `s2rebuild check` refuses a replacement or retained name that already exists.
- Pipelines the application recreates: list them in `databases[].app_managed_pipelines`. They are stopped at O2, never replayed, and C1 reports whether they came back.
- Configuration changed after O1: O1 records the configuration hash. Every command that opens the run journal refuses a configuration that hashes differently, until `--accept-config-change NOTE` records both hashes and the note in the journal. A changed database name is refused with or without it: use the configuration of the run. `s2rebuild status` reports both.
- An admin step stopped at a check: the error names the check, the expected value and the observed value. Fix the cause and rerun the step; the freeze and the switch re-enter from the live attachment state.

Documents

- docs/STEPS.md: every step in full.
- docs/CONFIG.md: every key, flag, variable and exit code.
- docs/CODE-GUIDE.md: modules, data flow, decisions you can change.
- docs/LESSONS.md: Helios facts and run lessons.
- docs/ARCHITECTURE.md: the design.
- `docs/s2rebuild-guide.pdf`: this README and STEPS with the flow diagram, built by `docs/build_pdf.py`.

Steps

Author: John Bergamini

One section per step. Each lists the purpose, the identity, inputs, outputs, the statements the kit issues, the gate, how to re-enter and what to do when it fails. Admin steps print their block (also written to `customer/runs/<run>/<db>/owner-sql/<step>.sql`, mode 0600); with the owner profile the kit executes it statement by statement and stops at the first check that does not return the expected value. Every statement passes the step's guard first.

Commands below assume `s2rebuild` on the path and `customer/config.yaml` as the configuration. Add `--db NAME` when several databases are configured.

A step that passed is skipped on a rerun only while every step before it is passed. When an earlier step is reopened (a re-copy reopens O6, O7 and O8), the passed step is refused like a step that never ran. The refusal names the steps to pass first, earliest first, and how: `s2rebuild gate ID --pass` for a step a person records, `s2rebuild run --from ID` for a step the kit runs. Follow them in that order, since `run --from` stops at a gate a person has not recorded.

The same rules hold for every command that runs a step: `run`, `step`, `check` (P2) and `profile` (P3, O4).

1. A step whose earlier steps are not passed is refused for every selected database before the command connects or writes. Gate: the refusal names the steps to pass. A dry run reports it and goes on.
2. `profile` captures a passed step again only with `--redo`, and the log names what it rewrites. `check` reruns a passed P2 without it.
3. `gate` records a step whose earlier steps are not passed, with a warning that names them. Gate: the warning is printed and logged.
4. O1 records the database names and the configuration hash. Every command that opens the run journal refuses changed names. It refuses a changed configuration unless `--accept-config-change NOTE` records an event with both hashes and the note. Gate: `s2rebuild status` shows no warning. A dry run and `status` report a changed hash and go on.
5. Every command works on the run that `--run ID` or `S2REBUILD_RUN` names, else on the most recent run. `run` resumes that run and skips the steps that passed. It starts a run only when none exists, or with `--new` (`--new --run ID` names it; an id that exists is refused). Once a run exists, `run --run ID` refuses an id that names no run. Every gate hint and every log header prints the run id with its `--run` form. Gate: one run directory holds P1 to O11 of the night.

P1 Prerequisites

- Purpose: an account for the copy, its password stored, the configuration written.
- Identity: owner (account) and the operator.
- Inputs: `examples/config.example.yaml`, the admin.
- Outputs: the runner account with the privileges of section 5 of the architecture; a Keychain item or variable per identity; `customer/config.yaml`.
- Statements: `s2rebuild account-sql` prints CREATE USER and GRANT with checks before and after; `--execute` runs them with the owner profile and injects the password at execution only.
- Gate: the runner account, the Keychain items (or variables) and the configuration are in place. Record: `s2rebuild gate P1 --pass --note TEXT`. When no run exists yet, this gate starts the run and prints its id; export it as `S2REBUILD_RUN`. Once a run exists, `--run ID` must name it.

- Re-enter: rerun `account-sql`; its first check stops when the account already exists.
- When it fails: a missing privilege is named by `check` (P2); grant it and rerun `check`.

P2 Preflight

- Purpose: prove the connection, privileges, layout, storage id, names and compute size before anything else.
- Identity: runner, plus the owner when configured.
- Inputs: the configuration; P1 recorded in the run. Until then `check` refuses and creates nothing.
- Outputs: the source STORAGE_ID in the journal, the engine version, a preflight record.
- Statements: SELECT only: CURRENT_USER and version, USER_PRIVILEGES for the runner, DISTRIBUTED_DATABASES, MV_CLOUD_ATTACHED_DATABASES, ROUTINES, MV_NODES leaf cores, PIPELINES count (owner).
- Gate: every line ok: all runner privileges present, source reported with a STORAGE_ID, every procedure and function of the source listed in `objects.routines.ignore` (routines are not migrated in this pass), replacement and retained names free, leaf cores times 8 at or above the target partitions (when MV_NODES is readable; otherwise confirm in the Portal).
- Re-enter: `s2rebuild check` any time; a passed P2 runs again without `--redo` and records its result again. O1 needs P2 passed in the same run.
- When it fails: each failed line says what to change (grant, rename, compute size). A routine line names each unlisted procedure and function: list it by name (every database) or as `database.name` (one database) under `objects.routines.ignore` once its owner accepts that the live database has no such routine after the switch, then run `s2rebuild check` again.

P3 Profile

- Purpose: know the database: tables, keys, sizes, accounts, sessions, views, routines, pipelines; estimate the copy.
- Identity: runner; owner for pipelines.
- Inputs: the configuration, `customer/baseline.json` when present; P1 and P2 passed in the run.
- Outputs: `inventory/P3/inventory.json`, `tables.csv`, `definitions/`, `pipelines/` (0600, CREDENTIALS replaced by <<CREDENTIALS>>), `reports/profile.xlsx` with the estimate, bands and window fit.
- Statements: SELECT and SHOW only (SHOW CREATE PIPELINE ... EXTENDED with the owner profile).
- Gate: every routine, and every procedure a pipeline loads into, is listed in `objects.routines.ignore`; the estimate at the high band fits before the abort deadline. Listed routines show as not migrated on the plan sheet and the workbook Summary.
- Re-enter: once P3 passed, `s2rebuild profile --redo` captures again; the log names what it rewrites. `--exact-counts` for COUNT(*) per table. `--dry-run` writes only `reports/profile-dry-run.xlsx` (or `--out`), never the capture under `inventory/`.
- A stop part way (a lost connection, Ctrl-C) leaves P3 not passed; a plain `s2rebuild profile` captures again.
- When it fails: `DOES NOT FIT` exits 4; widen the window, raise workers, or split the databases across nights (`--all-databases` ranks them). `P3 refused` exits 1, records P3 failed and writes no workbook; the refused capture stays under `inventory/P3/` for review and the plan sheet. It names each unlisted

routine and each pipeline into an unlisted procedure; list them as for P2 and run `s2rebuild profile` again. A pipeline into a listed procedure fails at O9 unless that procedure exists in the replacement. A pipeline with an empty definition, or a CREDENTIALS clause the kit cannot replace, refuses the capture by name and nothing is written; run `s2rebuild profile` again.

P4 Plan

- Purpose: the decisions people make before the night.
- Identity: the people named in the plan.
- Inputs: `s2rebuild plan`.
- Outputs: a recorded gate.
- Statements: none.
- Gate: freeze path, window, abort deadline, the named owner for "application stopped", the pipeline credential route and handover, the retained name and retention days, recorded in the configuration.
- Re-enter: edit the configuration and print the plan again.

P5 Rehearsal

- Purpose: run the exact steps on throwaway databases first, including a hard kill and the switch.
- Identity: runner and owner.
- Inputs: `--rows` (default 1000000), `--prefix` (must start with `s2rebuild_rehearsal_`).
- Outputs: `customer/runs/<run>/rehearsal/timings.json` with every step time, the interrupted and reconciled chunks and the Portal cleanup note.
- Statements: CREATE and DROP DATABASE on the prefixed names only, the copy, verify and switch statements on them.
- Gate: P5 passed: copy, SIGKILL and resume (exactly the killed chunk reconciled), the view created by the owner at O7, verify, freeze and switch completed. Without the owner profile O7 ends pending, the view and the switch are not rehearsed, and P5 stays pending.
- Re-enter: `s2rebuild rehearse --cleanup --prefix PREFIX --yes`, then rehearse again.
- When it fails: the kit drops only the databases it created in that run.

O1 Start

- Purpose: start the window with a recorded go decision.
- Identity: runner and the operator.
- Inputs: the window in the configuration; P2 passed in this run; rehearsal evidence: P5 passed in this run, or a rehearsal of any run under `customer/runs/` that records P5 passed.
- Outputs: names, window, abort deadline (end minus restart allowance, buffer and the longest modeled chunk) and the configuration hash in the journal.
- Several databases: `s2rebuild run` runs O1 for every selected database, then stops once for the go. `s2rebuild gate O1 --pass` records the go in every journal. A refused or failed O1 stops the run at once.
- Statements: none beyond reading the capture.
- Refusal: without P2 passed in this run, or without rehearsal evidence, O1 refuses before it records the window, the names or the configuration hash; its note says why. Run `s2rebuild check` in this run, or

`s2rebuild rehearse`. To start without a rehearsal, run `s2rebuild run --from O1 --no-rehearsal NOTE`; the note is recorded as an event and in the O1 note.

- Gate: deadline in the future, P2 passed in this run, rehearsal evidence present or waived, go recorded: `s2rebuild gate O1 --pass --note TEXT`. The note records whether the application stays stopped until O11. Only a stopped application keeps the O10 rollback possible after S3.
- From O1 on: a changed configuration is refused by every command until `--accept-config-change NOTE` records it; a changed database name is refused with or without it.
- A go recorded in a journal where O1 never ran is recorded with a warning: the checks and records of O1 are missing there. Run `s2rebuild run --from O1 --to O1 --redo`, then record the go again.
- Re-enter: `s2rebuild run` resumes the same run. It skips O1 once the go is recorded and continues at O2. `s2rebuild run --new` starts another run instead, which needs its own P1 and P2.

O2 Stop pipelines

- Purpose: stop every pipeline and record the final bookmark of each.
- Identity: owner.
- What a pipeline and an offset are: a pipeline is a server-side ingestion job reading a source (a Kafka topic, an S3 prefix and others) into one table. Its offset is its bookmark per source partition. A pipeline recreated with its recorded offsets resumes where the old one stopped: no message skipped, none loaded twice.
- Outputs: `inventory/O2/` with every definition (credentials replaced by the placeholder) and the offsets read after the stop; journal pipeline rows.
- Statements: STOP PIPELINE per pipeline (skipped only for one already Stopped, so a pipeline in state Error is stopped too), a check that none is Running, then the capture. The stop list is the earlier capture plus any pipeline the live view shows.
- Gate: no pipeline Running; every captured pipeline Stopped; LATEST_LOADED_OFFSET equals LATEST_SUCCESSFUL_OFFSET on every partition, since O9 replays the committed position. A pipeline that runs again before the capture refuses the gate. A partition where the two offsets differ refuses it too, a NULL successful offset beside a loaded one included, with the partitions listed; rerun O2 once they agree. A pipeline whose SHOW CREATE PIPELINE returns no CREATE statement, or whose CREDENTIALS clause has a form the kit cannot replace, refuses the capture by name and nothing is stored; rerun O2. A topic, table or column named credentials does not refuse it.
- Notebook route: the O2 cell's result carries the capture; `s2rebuild gate O2 --pass --note TEXT --evidence FILE` imports it.

O3 Freeze

- Purpose: make the source read only for every account.
- Identity: owner; the write probe runs as the runner.
- Path A (default): checks (READWRITE, recorded STORAGE_ID, no pipeline Running), DETACH, check detached, ATTACH ... STORAGEID ... READ ONLY, checks (READONLY, same STORAGE_ID), then the write probe: a DELETE for a value that cannot exist on a string column (or an integer key below its minimum). Pass means the server refused it as read only. Then freeze_time and, with the owner, a grants reference for C1.
- DETACH waits: after `freeze.detach_wait_seconds` the kit cancels the waiting DETACH on its own connection and issues DETACH ... KILL when `freeze.detach_kill` is true; otherwise it stops and the lockout serves as a bridge.

- Path B: `s2rebuild lockout`: snapshot every write-capable account at every scope (global, database, table) to the journal and `grants-snapshot.json` (0600), revoke exactly those privileges, hold connections, and on every pass re-revoke grants restored by another session and kill every session of those accounts (idle ones too) from the connection that sees it. After the configured quiet passes the quiet proof runs; it records freeze_time when pipelines are stopped, no foreign session is active and counts and newest values hold for a minute.
- Path B is single database: with more than one configured database, `check`, `run`, `step` O3, O10 and O11, `lockout` (with or without `--hold`), `owner-sql` O3, O10 and O11, `owner-notebook`, and `gate` O3, O10 and O11 `--pass` refuse. They write no block, record no gate, take no snapshot and revoke nothing.
- Path B hold: start it as `s2rebuild lockout --hold` in a second terminal and leave it running through O10. After the quiet proof it keeps passing (re-revoke, kill, one line per pass) and writes each pass to the journal. It stops by itself once S1 of O10 has run, or on `--until`, the stop file or Ctrl-C. O10 refuses at S0, before the go, while the last pass is missing, older than `freeze.quiet_max_age_seconds` or not quiet. The hold needs the owner profile; on the hand and notebook routes the O10 forward block checks the privileges live instead (O10 Switch, S1).
- Gate: Path A, probe refused and READONLY; Path B, quiet proof passed.
- Re-enter: `s2rebuild step O3 --redo`. When the source is already READONLY on the recorded STORAGE_ID the kit only checks, probes and records freeze_time, with or without the owner profile.
- Re-enter between DETACH and ATTACH: when no database carries the source name and the recorded STORAGE_ID shows IS_DETACHED = 1, `s2rebuild step O3 --redo` prints the re-entry block to `owner-sql/O3-reentry.sql` and runs it with the owner profile: checks (storage detached, no database named source or replacement), ATTACH ... STORAGEID ... READ ONLY, the READONLY checks, then the probe and freeze_time. It never detaches again. The kit reads this position with the owner session when the owner profile is configured, otherwise as the runner. Without the owner profile, run the printed block, then `s2rebuild step O3 --redo`. The notebook O3 cell runs the same block (`O3-reentry`) when the whole block stops at its first check with no rows. A database under the source name on another STORAGE_ID still refuses at the first checks.
- Hand route: `s2rebuild owner-sql O3` writes and prints both blocks, `owner-sql/O3.sql` and `owner-sql/O3-reentry.sql`. The whole block opens with a note that says when to run the re-entry block instead: its first check finds no rows. Gate: the re-entry block's checks pass; a database under the source name stops it before the ATTACH.

O4 Freeze capture

- Purpose: the inventory the copy uses, taken after the freeze.
- Identity: runner.
- Inputs: `s2rebuild profile --step O4`, after O1 to O3 passed. Until then it refuses before it connects or writes.
- Outputs: `inventory/O4/`, `drift.json` against P3, the copy plan.
- Gate: drift reviewed; the routine rule of P3 holds again, so a routine created since P3 and not listed in `objects.routines.ignore` refuses O4 and records it failed; O5 and later refuse until O4 passes.
- Re-enter: once O4 passed, `s2rebuild profile --step O4 --redo` captures again and rewrites the copy plan; the log says so. O5 and O6 refuse a plan that differs from the journal. A stop part way leaves O4 not passed; a plain rerun captures again. A dry run writes nothing under `inventory/`.

O5 Create replacement

- Purpose: the replacement database with the target layout and identical table definitions.
- Identity: runner.
- Statements: CREATE DATABASE ... PARTITIONS ... SUB_PARTITIONS, CREATE TABLE ... LIKE per table, SHOW CREATE TABLE.
- Outputs: the replacement STORAGE_ID in the journal (the switch attaches it by that id).
- Gate: layout and durability verified; every definition equals the capture (only the AUTO_INCREMENT counter may differ).

O6 Copy

- Purpose: copy every row.
- Identity: runner.
- Statements: INSERT INTO replacement SELECT FROM source WHERE CRC32(key) MOD B = k per bucket; reconciliation DELETE of a started or failed bucket before it is re-inserted.
- Behaviour: a queue per table; a table that hits a lock wait (1205) is serialized and probed; the journal is written before each statement; the abort deadline and `s2rebuild pause` stop new chunks while running ones finish. A chunk fails at its `copy.max_lock_waits`-th lock wait, and the O6 note names `s2rebuild step O6 --fresh-table TABLE`. Ctrl-C stops new chunks; a second Ctrl-C only prints that a hard stop needs kill, and the workers still finish their chunks. Never hard-stop the copy: a killed client leaves a prepared transaction holding a leaf table lock for 15 to 20 minutes.
- Gate: every chunk done; per-table count proof all match.
- Re-enter: `s2rebuild run --from O6` resumes; `--retry-failed` reruns parked chunks; `s2rebuild step O6 --fresh-table TABLE` loads a locked table into a new table and renames it. The fresh-table load refuses past the abort deadline, while the stop file exists, and when the fresh name would be over 64 characters or a planned table. It reuses only a fresh table its journal recorded; another table under that name refuses, and a missing replacement table is never recreated from the source. After a failure, rerun the same command: it resumes from the recorded fresh table and does not load it again once it is complete. After a re-copy mark, or a copy run that finished the table, it loads the table again. The fresh-table load replaces the whole table, so before its first statement it clears the table's verification and sets O7 and O8 back to pending when they passed, and the O10 block files get a STOP as for a re-copy. After it, run `s2rebuild run --from O6`: O7 sets the counters of the new table, O8 verifies it.
- Re-copy: `s2rebuild mark-recopy TABLE [--bucket N]` marks buckets for the next copy run, which deletes and re-inserts them. In the same journal write it sets O6, O7 and O8 back to pending when they passed, and clears the table's `verified_at_utc` and `target_count`. With several databases it checks every selected journal first, so a refusal for one marks none. Just before that journal write it rewrites `owner-sql/O10.sql` and `owner-sql/O10-rollback.sql`, when they exist, so each starts with a STOP, and it says that a notebook generated before must not run its O10 cells. An O7 or O8 re-copy mark and a fresh-table load do the same. Then:
  1. Run `s2rebuild run --from O6`. O6 copies the marked buckets again. Gate: O6 passed.
  2. O7 runs again: counters after the re-copy, then the stale check. Gate: O7 passed.
  3. O8 runs again on the re-copied rows. Gate: O8 passed. Until then S0 refuses the switch and the rollback block stops.

4. Before a hand or notebook switch, generate the O10 blocks again: `s2rebuild owner-sql O10` and `s2rebuild owner-sql O10 --rollback`, or `s2rebuild owner-notebook`, and use only the new notebook. Gate: the forward block holds no STOP.

- Re-copy after the switch began: refused. Once O10 passed, the go decision is recorded, an O10 gate is recorded (pass or fail), a refused `gate O10 --pass` recorded the switch its evidence shows, or the switch time or the retained or live attachment is recorded, `mark-recopy` and O6, O7 and O8 refuse with nothing written. A passed O6 to O8 is still skipped, so `s2rebuild run` continues to O11. A new copy is a new run.
- An O10 that stopped before the go decision issued nothing, so it does not block a re-copy: a Ctrl-C at the typed-name prompt, a lost owner connection, a stop during S0. A refusal at S0 or at an S1 before-check does not block it either: those checks run before the go, the Path B lockout pass among them. When O10 reads a switch already under way (a hand S1, say), it records that before the go, and the re-copy stays refused. After a hand or notebook switch, record it with `s2rebuild gate O10 --pass` or `--fail`, so the journal knows.
- While O10 runs, it holds the copy lock of the journal. `mark-recopy`, O6 and an O7 re-copy round in another terminal refuse until O10 ends, with nothing marked or copied; the refusal names the process. As O10 records the go decision, it reads the journal part of S0 again in the same journal write. A re-copy mark (an O8 mismatch, say) or O6 to O8 started in another terminal while the operator typed the live name refuses the go, with nothing issued. Then run `s2rebuild run --from O6`, and O10 again once O8 passed.

O7 Post-copy

- Purpose: views, counters and the stale check.
- Identity: runner; owner for the views. A view runs under its definer and C4 removes the runner, so the runner never creates a view.
- Statements: USE and CREATE VIEW naming the replacement (owner), ALTER TABLE ... AUTO_INCREMENT and AGGREGATOR SYNC AUTO_INCREMENT ... ALL, counts and newest values on both sides.
- Source names: a view definition that names the source database refuses the step before any view statement. Edit the definition in the capture so it names no database, then rerun O7.
- Views with the owner profile:
  1. The owner session creates each missing view. It keeps the original DEFINER when that account still exists and is not the runner. Otherwise it removes the clause and the owner account becomes the definer. The journal and the Views tab of the final workbook record the choice per view.
  2. A view that fails is tried again after the others, in rounds, so a view on another view is created once the other exists. Gate: every view created. A round that creates none refuses the step with each view and its error number; fix the cause and rerun O7. Views already created are skipped.
- Views without the owner profile:
  1. O7 prints the O7 block and writes it to `owner-sql/O7.sql`: every view in dependency order, each skipped when it exists and checked after. The admin who runs it becomes the definer.
  2. The stale check and the counters run. O7 ends pending.
  3. Run the block as the admin, then `s2rebuild step O7`. Gate: the rerun finds every view and passes.
- Counters: each AUTO_INCREMENT counter is set to the larger of the source counter and MAX + 1, then synced on every aggregator (ALL). An empty table without a source counter keeps the default. The table tab of the final workbook shows both values and the decision. A rerun of O7 sets and syncs every counter

again.

- Re-copy: a stale mismatch marks the table for re-copy, the kit copies the marked buckets again and repeats the check, for up to three rounds. A round that ends with every chunk done and the count proof all matching marks O6 passed again. The counters are set once, after the last round.
- Gate: views created, counters set, zero stale mismatches after re-copy.
- When it stops: a re-copy round that does not complete leaves O6 pending; run `s2rebuild run --from O6`, which finishes the marked buckets and reruns O7.

O8 Verify

- Purpose: prove the copy.
- Identity: runner.
- Tiers: 1 counts both sides; 2 newest time value and key range; 3 grouped fingerprints (key hash and value hash per group, mapped to buckets), smallest tables first within `verify.fingerprint_budget_minutes`, sampled beyond it.
- Outputs: verification rows, each with the table's re-copy epoch, `verified_at_utc` per table, the final workbook and run report.
- Gate: tiers 1 and 2 pass everywhere; tier 3 within budget, or a sampled result accepted with `s2rebuild step O8 --yes`.
- When it fails: exit 3; the mismatching buckets are marked for re-copy, which reopens O6 and O7. Run `s2rebuild run --from O6`: O6 copies them again, O7 resets the counters, O8 verifies again.

O9 Pipelines

- Purpose: every pipeline exists in the replacement, stopped, at the offsets recorded at O2.
- Identity: owner.
- Credentials: the server hides CREDENTIALS from every identity, so the customer supplies them. Route kit: the operator exports the variable per source type (or per pipeline) named in `pipelines.credentials_env` and `pipelines.per_pipeline_env` for this step only and unsets it afterwards; the kit injects the value as a SQL literal at execution and masks it in every log line. In the notebook the cell asks for each value with a hidden prompt. Route customer: the customer runs `pipelines/recreate-with-placeholders.sql` from the O2 capture inside the replacement, filling in CREDENTIALS; the kit runs only the checks.
- Statements: USE replacement, CREATE PIPELINE per pipeline (skipped when it exists), its ALTER PIPELINE ... SET OFFSETS statements. Never START.
- Gate: count equals the capture minus application-managed pipelines; none Running; the committed offset of every partition in the replacement equals the O2 capture. The committed offset is LATEST_SUCCESSFUL_OFFSET; O2 refuses a capture that holds a loaded offset without a successful one. The block checks each partition, so the notebook and hand routes hold the same gate.
- Refusals: only the O2 capture is replayed (an earlier capture's offsets were still moving); a definition that names the source database is refused. A definition that still carries a redaction marker (a value the server masked) is refused by name before any statement, by the kit and by the notebook cell; capture it again.

O10 Switch

- Purpose: the live name resolves to the replacement, the original stays readable.
- Identity: owner.
- S0 (decision): O8 and O9 passed; every planned table verified after freeze_time and after its last chunk, at its current re-copy epoch (a table without a stamp refuses, by name); both storage ids recorded; time left before the abort deadline; the operator types the live name (or passes `--yes`). The go is recorded only while the journal part of S0 still holds, and O10 holds the copy lock from S0 to S3 (O6 Copy).
- S0 on every route: without the owner profile O10 refuses on the journal part of S0 (O8 and every step before it passed, freeze_time, every table verified) and prints the forward block with a STOP first. `s2rebuild owner-sql O10` and the notebook O10 cell carry the same STOP while it holds, and `s2rebuild gate O10 --pass` refuses. Generate the forward block or the notebook after O8 passed, and again after a re-copy (O6 Re-copy, step 4).
- A forward block generated before a re-copy: the kit rewrites the files under `owner-sql/` with a STOP, but cannot reach a notebook already uploaded. When `gate O10 --pass` refuses and its `--evidence` shows the switch ran (the retained and live attachments), the kit records those attachments and the pass stays unrecorded. From then on a re-copy is refused. The refusal names the rollback block. Its proof needs the copied row counts; when a re-copy cleared them, the block stops and the way back is a reconciliation. Without evidence, record what ran with `s2rebuild gate O10 --fail --note TEXT --evidence FILE`.
- When S0 refuses: the refusal names each table and the way to verify again. `s2rebuild step O8 --redo` when O7 passed after the last chunk; `s2rebuild run --from O6` while O7 is pending after a re-copy.
- S1: checks (source on its recorded STORAGE_ID, attach type for the freeze path, no foreign active session, replacement READWRITE on its recorded id, retained name free; on Path B every write privilege the lockout took still revoked, one check per account and scope, and the last lockout pass quiet within `freeze.quiet_max_age_seconds`), DETACH (with the wait and KILL rule), checks. The kit runs the before-checks once before the go and again before the DETACH. The Path B pass check reads the journal, so the printed block and the notebook carry it as a note; their privilege checks stop them before the DETACH when a grant came back after the quiet proof. Revoke it again as the O3 lockout plan does, kill that account's sessions, and run the block again from the top. Generate the block or the notebook after O3 recorded the snapshot; without one the block says it has no privilege check.
- S2: ATTACH source AS retained STORAGEID ... READ ONLY, checks, one small table count.
- S3: checks, ATTACH replacement AS live STORAGEID ..., checks: READWRITE on a new STORAGE_ID, target layout, table, view and pipeline counts, three smallest tables against the journal.
- Re-enter: the position comes from the attachment views. A rerun repeats the after-checks of the step it lands behind, so a failed check never turns into a passed switch.
- Write boundary: S3 attaches the live name READWRITE, so on Path A the application can write from S3, before O11.
- Rollback (before O11 only): the zero-change proof when S3 ran, R1 detach the live branch, R2 re-attach the original read write by STORAGEID, R3 drop the retained copy (optional). It resumes from any position behind S1; behind S3 it starts at R2 without the proof.
- Zero-change proof: every planned table on the live name holds the row count of the copy, no pipeline is Running on it, no write privilege the lockout took is back, and no foreign non-idle session is active on it. It detects changed row counts and active writers only, not an update that keeps the count. After S3 a

rollback is safe only when the O1 note records that the application stayed stopped.

- Rollback block: `s2rebuild owner-sql O10 --rollback` prints it, and the notebook has an O10 rollback cell. Both carry the proof as checks before R1. Generate the block or the notebook (`s2rebuild owner-notebook`) after O8 passed, so it holds the copied row counts. Generate it again just before a rollback, so it holds the O11 state.
- Behind S3 the notebook cell stops at the first proof check, with nothing issued. Finish by hand from the R2 check of the printed block.
- Rollback refused, before any statement: once O11 started releasing writers, once C1 saw application writes, or when the proof fails. The kit knows that O11 started from release_started_utc, released_utc or the O11 status. After a hand or notebook O11 the journal may not know, so the proof refuses on the live state: a pipeline Running or a privilege back. A failed proof lists what differs and says reconcile first: a new freeze and a reconciliation, never R1.

O11 Release

- Purpose: writers back on the live name.
- Identity: owner and the customer.
- Retention check: for every captured partition the recorded committed offset must not be below the current EARLIEST_OFFSET, and the partition must exist. A gap means messages were lost to the source's retention; decide EARLIEST or LATEST per pipeline and record it in the gate note.
- Statements: GRANT restores (each checked) from any unrestored lockout snapshot, START PIPELINE per expected pipeline (skipped when Running), SET GLOBAL query_parallelism_per_leaf_core when configured.
- Release start: with the owner profile the kit records release_started_utc in the owner session, once the retention check and every check ahead of the first statement passed, just before that statement. From then on a rollback is refused, even when O11 stops part way. An O11 that stops before its first statement (a failed check, a lost connection) records nothing, and the rollback stays open. A hand or notebook O11 records nothing; there the rollback proof refuses once a pipeline runs or a privilege is back on the live name.
- Gate: every expected pipeline Running; grants equal the snapshot; the application restarted and writing. The kit stamps released_utc when every check passed; recording the gate by hand stamps it too.
- Re-enter: `s2rebuild step O11` after a failure. A pipeline already Running is skipped, so no START is issued twice.

C1 Verify after release

- Identity: runner; owner for pipelines and grants.
- Checks: writer watch, count changes on the largest tables (two samples a minute apart and against the copy), pipelines Running with offsets advancing where data waits, application-managed pipelines present again, no pipeline errors since the release, grants equal to the snapshot (Path B) or the reference (Path A). Application writes are recorded; from then on a rollback is a new freeze and a reconciliation.
- Gate: every check ok and application writes seen on the live database (a session, counts moving, or counts that differ from the copy). A live database that is idle on purpose passes with `s2rebuild step C1 --idle-workload NOTE`; the note is journaled. Without writes or the flag C1 fails with exit 4. A missing application-managed pipeline fails C1. Without the owner profile the pipeline and grants checks are recorded as not checked, no owner profile, and C1 passes with that note.

- Re-enter: `s2rebuild step C1`. It samples again; writes made earlier still show as counts that differ from the copy, and writes an earlier C1 attempt of the run saw (journal `app_writes_seen`) still count when the new sample sees none, with the time named in the note.
- When it fails: fix what the note names and rerun. When the kit cannot see the evidence, record C1 by hand with `s2rebuild gate C1 --pass --note TEXT --override NOTE`. Gate refuses C1 without `--override`; the override is journaled.

C2 Report and calibrate

- Identity: runner.
- Outputs: `reports/final.xlsx`, `reports/run-report.md`, `customer/baseline.json` refitted from the run's first-pass chunks when the journal can be fitted.
- Gate: reports written. Calibration is advisory: with too few chunks to fit (or a fit the model refuses) the baseline in use stays, the Summary of both reports carries a Calibration line that says so, and C2 passes with that note.
- Re-enter: `s2rebuild step C2` writes the reports and the baseline again from the same journal.
- When it fails: fix the cause and rerun. To record C2 by hand: `s2rebuild gate C2 --pass --note TEXT --override NOTE`. Gate refuses C2 without `--override`; the override is journaled.

C3 Retention cleanup

- Identity: owner.
- When: after `closeout.retain_days` from the release; the step refuses earlier.
- Statements: a check that the live name resolves to the live branch, the identity checks, DROP DATABASE retained, DROP DATABASE replacement, after each DROP checks that the name is free and that the recorded STORAGE_ID is attached nowhere, and a last check that the live name still resolves to the live branch.
- Identity checks: a DROP runs only while its name holds the database the run recorded: the retained branch with the STORAGE_ID recorded at O10 S2, READONLY; the replacement with the STORAGE_ID recorded at O5, READWRITE. Each identity is two checks: the name holds the recorded database or nothing, and the recorded STORAGE_ID is attached under no other name. Both are checked before the first DROP; the retained check sits directly before its DROP and the replacement is checked again just before its own DROP. A STORAGE_ID that was never recorded refuses the step before any statement.
- Gate: pass when both DROPs ran or were skipped, neither recorded STORAGE_ID is attached anywhere, and the live name resolves to the live branch.
- Re-enter: `s2rebuild step C3`. A DROP is skipped unless its name holds the recorded database, so a rerun after a partial cleanup drops only what is left. The printed block and the notebook cell carry the same checks and skips.
- When it fails: another database under either name, the retained branch attached READWRITE, the replacement attached READONLY, or a recorded STORAGE_ID attached under another name stops the block before the first DROP with nothing dropped. The last case means the database was moved, or the block was built from names the run did not record. A database that appears under a name after its identity check is skipped, never dropped, and the check after that DROP stops the block. Find out who attached the name; C3 never drops a database the run did not record.
- Then: delete the detached original storage in the Portal by STORAGE_ID; keep the evidence folder.

C4 Remove the migration account

- Identity: owner.
- Statements: a check that the account still holds privileges, a check that no view in the live database has the runner as its definer, DROP USER, a check that zero privileges remain.
- When it stops at the view check: a view whose definer is gone can no longer be executed. List them with `SELECT TABLE_NAME, DEFINER FROM information_schema.VIEWS WHERE TABLE_SCHEMA = '<live database>'`, recreate each one the runner defines as a durable account (the admin or the application account), then rerun `s2rebuild step C4`.
- Then: delete the Keychain item that held the runner password.